

AI 狼人杀 Agent Team — Vibe Coding 开发步骤

1. 定义项目目标

```
1  目标：
2  - 多 Agent 狼人杀系统
3  - 支持纯 AI 对战
4  - 支持人机混战
5  - 支持日志回放
6  - 支持不同模型 Agent
7  - 可扩展角色系统
```

2. 第一轮 Claude Code 对话（项目初始化）

Prompt

```
1  你现在是我的 AI 架构师。
2
3  帮我设计一个“AI 狼人杀多 Agent 系统”。
4
5  要求：
6  - Python
7  - FastAPI + WebSocket
8  - LangGraph / AutoGen 风格多 Agent
9  - 支持：
10 - 狼人
11 - 预言家
12 - 女巫
13 - 猎人
14 - 平民
15
16 需要：
17 1. 项目目录结构
18 2. 模块拆分
19 3. Agent 抽象接口
20 4. Game Engine 状态机
21 5. 回合流程
22 6. 日志系统
23 7. Prompt 管理
24 8. Memory 管理
25 9. 前后端通信协议
26
27 只输出：
28 - 文件树
29 - 类结构
30 - 数据流
31 - API 定义
```

3. 第二轮 Claude Code（生成项目骨架）

Prompt

```
1  根据刚才架构：
2
3  直接生成：
4  - 完整项目目录
5  - 所有空文件
6  - requirements.txt
7  - docker-compose.yml
8  - README.md
9  - .env.example
10
11 要求：
12 - 可直接运行
13 - 模块解耦
14 - 预留多模型接口
```

4. 第三轮 Claude Code（实现核心 Game Engine）

Prompt

```
1  实现：
2
3  GameEngine
4
5  要求：
6  - 状态机驱动
7  - 夜晚阶段：
8    - 狼人行动
9    - 预言家查验
10   - 女巫行动
11  - 白天阶段：
12   - 发言
13   - 投票
14   - 放逐
15  - 胜负判定
16  - Event Bus
17  - Observer Hook
18
19 输出：
```

- 20 - 完整代码
- 21 - 类型注解
- 22 - 单元测试

5. 第四轮 Claude Code（实现 Agent 基类）

Prompt

```
1 实现 Agent Base Class:
2
3 要求:
4 - 独立 memory
5 - 独立 prompt
6 - 独立身份视角
7 - action()
8 - speak()
9 - vote()
10 - reflect()
11
12 支持:
13 - OpenAI
14 - Claude
15 - DeepSeek
16
17 要求:
18 - strategy pattern
19 - async
20 - token usage logging
```

6. 第五轮 Claude Code（实现角色 Agent）

Prompt

```
1 基于 Agent Base:
2
3 实现:
4 - WerewolfAgent
5 - SeerAgent
6 - WitchAgent
7 - HunterAgent
8 - VillagerAgent
9
10 要求:
11 - 每个角色有独立 system prompt
12 - 不同推理链
13 - 不同目标函数
```

- 14 - 不同决策逻辑
- 15 - 输出结构化 JSON

7. 第六轮 Claude Code（实现信息隔离）

Prompt

```
1 实现 Information Isolation Layer
2
3 要求：
4 - 不同 Agent 只能看到允许信息
5 - 狼人共享狼人视角
6 - 好人阵营信息隔离
7 - 历史事件裁剪
8 - Prompt 注入防护
9 - Memory 沙箱
10
11 输出：
12 - middleware
13 - validator
14 - policy engine
```

8. 第七轮 Claude Code（实现日志系统）

Prompt

```
1 实现完整日志系统：
2
3 要求：
4 - 所有 Agent 行为记录
5 - Prompt 记录
6 - 模型输出记录
7 - token 消耗记录
8 - 回合状态记录
9 - JSONL 格式
10 - Replay 支持
11
12 输出：
13 - logger.py
14 - replay_engine.py
```

9. 第八轮 Claude Code（实现 Web UI）

Prompt

```
1 实现观战 UI:
2
3 技术:
4 - Next.js
5 - Tailwind
6 - WebSocket
7
8 功能:
9 - 实时显示回合
10 - 玩家头像
11 - 发言气泡
12 - 投票动画
13 - 死亡动画
14 - 阵营显示
15 - Replay 回放
16
17 风格:
18 - 黑暗狼人杀风格
```

10. 第九轮 Claude Code（实现 AI 发言增强）

Prompt

```
1 优化 Agent 发言:
2
3 要求:
4 - 更像真人
5 - 有情绪
6 - 有误导
7 - 有博弈
8 - 会甩锅
9 - 会带节奏
10 - 会伪装
11 - 会撒谎
12
13 加入:
14 - Personality System
15 - Emotion State
16 - Trust Score
17 - Suspicion Score
```

11. 第十轮 Claude Code（实现评测系统）

Prompt

```
1 实现 Evaluation System
```

```
2
3 要求：
4 - 胜率统计
5 - 发言质量评分
6 - 推理正确率
7 - 生存率
8 - MVP 评分
9 - 对局复盘
10 - 不同模型对战 Benchmark
11
12 输出：
13 - leaderboard
14 - evaluator
15 - replay analyzer
```

12. 第十一轮 Claude Code（实现自进化）

Prompt

```
1 实现 Self-Evolving Agent
2
3 循环：
4 对局
5 → 分析
6 → 反思
7 → Prompt 优化
8 → 策略优化
9 → 再对局
10
11 要求：
12 - 自动保存最佳 Prompt
13 - ELO Rating
14 - 多版本 AB Test
15 - Genetic Prompt Mutation
```

13. Claude Code Debug Prompt 模板

```
1 你现在是资深 Python 系统架构师。
2
3 不要解释。
4
5 直接：
6 1. 找 bug
7 2. 分析 root cause
8 3. 修改代码
9 4. 输出 patch
10
```

- ```
11 要求：
12 - 最小修改
13 - 不影响现有架构
14 - 保持类型安全
```

---

## 14. Claude Code 性能优化 Prompt

---

- ```
1 分析当前系统瓶颈：
2
3 目标：
4  - 降低 token 消耗
5  - 降低 latency
6  - 提高并发
7  - 提高 replay 速度
8
9 输出：
10 - profiling
11 - flamegraph
12 - 优化方案
13 - patch
```

15. 最终 Demo 结构

- ```
1 demo/
2 |— backend/
3 |— frontend/
4 |— agents/
5 |— engine/
6 |— prompts/
7 |— memory/
8 |— logs/
9 |— replay/
10 |— evaluation/
11 |— evolution/
12 |— docker-compose.yml
```

---

## 16. 最终展示方式

---

- 1 演示:
- 2 - 6 AI 自动狼人杀
- 3 - GPT vs Claude vs DeepSeek
- 4 - 实时观战 UI
- 5 - Replay 回放
- 6 - Agent 思考过程
- 7 - 赛后复盘
- 8 - Leaderboard